Deformation Transfer for Multi-Component Objects

Pramook Khungurn

March 11, 2026

This article is written as I read the "Deformation Transfer to Multi-Component Objects" paper by Zhou et al. [ZXTD10].

1 Introduction

- This paper is a follow-up work to the classic "Deformation transfer for triangle meshes" paper by Sumner and Popovic [SP04].
- While not stated in the classic paper, the method only applies to meshes with a single connected component.
  - I personally tried to apply it to meshes with multiple components (the face of a VRoid model), and it turns out that, while each component might be deformed correctly, the relative positions among the components are just all over the place.
- This paper provides a solution to the limitation above.

2 Related Works

- The paper seems to have a collection of nice references, which I will add here.
- Gradient-domain mesh deformation.
  - The authors called the affine transformation computed for each triangle in the source mesh the *deformation gradient*.
  - The deformation gradient is transferred from the source mesh to the target mesh. Then, we solve a linear system (i.e., the Poisson equation) to smooth it out.
  - The authors thus call the process in the original paper *gradient-domain mesh deformation*.
  - There are several follow-up works in this direction [YZX+04, SCOL+04, BS08].
- There are a number of interesting follow-up works to the classic paper.
  - Zayer et al.propose to transfer gradients only at a few key points, and interpolate them to other points with a harmonic function [ZRKS05].
  - Botsch et al.shows that the transfer in the classic paper can be achieved without having to add an extra vertex to each of the triangles, making the linear system smaller [BSPG06].

3 Algorithm Overview

- The paper still relies on the user providing correspondence between the source and target meshes.
  - However, this time, each can have multiple components.
  - Correspondence can be provided between some pairs of components, but it is not guaranteed that all components will have them.
- The algorithm first computes a set of closest vertex pairs in the source (and target) mesh to build a graph that encodes the spatial relationship between components.
  - The correspondence and deformation transfer algorithms of Sumner and Popovic are then adapted to take into account this graph.
  - When establishing correspondence between the source and the target, the authors add an energy term to make the components without user-specified markers deform with the components that have them.
  - Not all target triangles may have correspondences with source triangles.
  - Moreover, some components in the target may not have any corresponding triangles in the source. These are called *orphan components*.
- The algorithm adds two new energy terms when deforming the target to match the source deformation gradient.
  - The first term tries to preserve the lengths of the graph edges so that the spatial relationship between components is maintained.
  - The second term is based on Laplacian coordinates [SCOL+04] and tries to preserve the surface details of the orphan components in the target.
  - The two newly added terms are not quadratic functions, and so cannot be optimized by a linear solve. The paper uses an iterative Gauss–Newton method [Khu24].
    - We need to find the Jacobian of the loss function, and we need to solve a linear system in each iteration.
  - The paper claims that this is efficient for all their experimental data.

4 The Algorithm

- Notations.
  - We are given the three meshes: the source mesh S, the deformed source mesh $\widetilde{S}$, and the target mesh T.
  - The target meshes can have multiple disconnected components $T = \{T_1, T_2, \dots\}$, and so are the source meshes.
  - We want to generate a deformed target mesh $\widetilde{T}$ that has the same topology as T. The relationship between T and $\widetilde{T}$ should be similar to the relationship between S and $\widetilde{S}$.
  - Let $\mathbf{u}_i$ denote a vertex in S and $\widetilde{\mathbf{u}}_i$ a vertex in $\widetilde{S}$. On the other hand, we let $\mathbf{v}_i$ and $\widetilde{\mathbf{v}}_i$ denote a vertex in T and $\widetilde{T}$, respectively.
  - We will indicate with a superscript the component that a vertex belongs to. For example, $\mathbf{v}_i^a$ is the same as $\mathbf{v}_i$, but a signifies that $\mathbf{v}_i$ belongs to the component T_a.
- Like in the work of Sumner and Popovic, we add a fourth vertex $\mathbf{u}_4$ to each triangle $(\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3)$ in S (and $\widetilde{\mathbf{u}}_4$ to each triangle in $\widetilde{S}$) in the direction of outward normal vector of the triangle, at a distance proportional to the square root of the area.

- The linear part of the affine transformation that maps $(\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3)$ to $(\widetilde{\mathbf{u}}_1, \widetilde{\mathbf{u}}_2, \widetilde{\mathbf{u}}_3)$ is given by

$$F = \widetilde{G} G^{-1}$$

where

$$G = \begin{bmatrix} \mathbf{u}_2 - \mathbf{u}_1 & \mathbf{u}_3 - \mathbf{u}_1 & \mathbf{u}_4 - \mathbf{u}_1 \end{bmatrix}$$
$$\widetilde{G} = \begin{bmatrix} \widetilde{\mathbf{u}}_2 - \widetilde{\mathbf{u}}_1 & \widetilde{\mathbf{u}}_3 - \widetilde{\mathbf{u}}_1 & \widetilde{\mathbf{u}}_4 - \widetilde{\mathbf{u}}_1 \end{bmatrix}.$$

These two matrices are 3×3 matrices. The translation part of the affine transformation is just $\widetilde{\mathbf{u}}_1 - \mathbf{u}_1$.

- The algorithm has the following big steps.

  1. **Proximity pair computation.**
     - For each pair of components (T_a, T_b), the *proximity pairs* P_{ab} is a selected set of vertex index pairs that represent key spatial adjacencies that subsequent processing will try to preserve during the transfer.
     -
     - P_{ab} might be empty if T_a and T_b are too far away from each other.
  2. **Establishing correspondence.**
     - We assume that the user has provided a small set of point-to-point correspondences between vertices in S and vertices in T.
     - The goal of this step is to establish triangle-to-triangle correspondences M:

       $$M = \{(s_1, t_1), (s_2, t_2), \ldots, (s_{|M|}, t_{|M|})\}$$

       where a pair (s_j, t_j) means that the triangle with index t_j in T should deform like the triangle with index s_j in S.
     - Unlike in the Sumner–Popovic paper, not all target triangles may have correspondences with source triangles.
     - We denote by H the set of "orphan" target triangles. (These are the ones that do not correspond to any source triangles.)
     - A component whose triangles do not have any correspondences is called an "orphan" component.
     - Let us denote the set of orphan components by T_H.
  3. **Deformation transfer.** The deformation is transferred by solving an energy minimization problem.

4.1 Proximity Pair Computation

- For every pair of components (T_a, T_b), find the shortest distance between them. This is the distance between vertex $\mathbf{v}_i^a$ in T_a and vertex $\mathbf{v}_j^b$ in T_b that is the smallest.

$$d_{a,b} = \min_{i,j} \|\mathbf{v}_i^a - \mathbf{v}_j^b\|.$$

Let us denote the pair of vertices that gives us $d_{a,b}$ by $(\mathbf{v}_{i_1}^a, \mathbf{v}_{j_1}^b)$.

- Build a complete graph G with the components as its nodes. The edges are weighted by the shortest distances computed above.

- Compute a minimal spanning tree K of the graph.

- For each component T_a, compute its largest distance d_a from each of its adjacent components in K

$$d_a = \max_{(a,b)\in K} d_{a,b}$$

- For every pair of components T_a and T_b not connected by an edge in K, if $d_{a,b} \leq d_a + \epsilon_a$ and $d_{a,b} \leq d_b + \epsilon_b$, connect T_a and T_b by adding an edge between them to K. Here, ϵ_a and ϵ_b are user specified thresholds.
- In the paper's definition, the paper defines ϵ_a (and ϵ_b) as the maximal edge length of the component T_a times 1.5.
  - By edge length, do they mean length of a triangle's edge?
- If T_a and T_b are not connected in K, then $P_{a,b}$ is empty.
- Otherwise, $P_{a,b}$ is the set of vertex pairs $(v^a_{i_k}, v^b_{j_k})$ such that $\|\mathbf{v}^a_{i_k} - \mathbf{v}^b_{j_k}\| \leq d_{a,b} + \min\{\epsilon_a, \epsilon_b\}$.

4.2 Establishing Correspondence

- Similar to what is done in the Sumner–Popovic paper, we try to match between S and T by deforming one mesh to the other.
  - However, this time, we deform T to S instead of S to T.
- Notations.
  - Recall that the vertices of T are denoted by $\mathbf{v}_i$.
  - The vertices of the deformed target $\widetilde{T}$ are denoted by $\widetilde{\mathbf{v}}_i$.
    * We will solve for these later. This is not the place we discuss them.
  - We will use the notation $\mathbf{x}_i$ to denote the vertices of T that are being deformed to be close to S.
    * This is to distinguish between the deformation in this section (T to S) and the one we will perform later (T to $\widetilde{T}$),
- The user provides us with a small set of correspondences M:

$$M = \{(s_1, t_1), (s_2, t_2), \ldots, (s_{|M|}, t_{|M|})\}.$$

  We require that, after the transformation of T to be close to S, the positions of the corresponding vertices should match exactly. In other words,

$$\mathbf{v}_{s_k} = \mathbf{x}_{t_k}$$

  for all $k = 1, 2, \ldots, |M|$.
- The correspondence is determined by minimizing the following *corresponding energy* with respect to the constraints above:

$$\begin{aligned} &\text{minimize } E = w_S E_S + w_I E_I + w_C E_C + E_R \\ &\text{subject to: } \mathbf{v}_{s_k} = \mathbf{x}_{t_k} \text{ for all } k = 1, 2, \ldots, |M|. \end{aligned}$$

- Here, the deformation smoothness term E_S, the deformation identity term E_I, the closest valid point term E_C and their weights are lifted directly from the Sumner–Popovic paper.
- The paper adds a new term E_R to make the components without any user-specified correspondences deform along with those that do. It is computed as follows.

- For each component pair (T_a, T_b),
  find a pair of triangles $(t^a_{i_k}, t^b_{j_k})$
  such that, for each vertex pair $(v^a_{i_k}, v^b_{j_k}) \in P_{a,b}$,
  $t^a_{i_k} \in \mathcal{N}(v^a_{i_k})$ and $t^b_{j_k} \in \mathcal{N}(v^b_{j_k})$
  (where $\mathcal{N}(v)$ denotes the topological one-ring around v)
  and that the triangle has the largest separation along the normal directions.
  In particular, the pair is defined as

$$\underset{\substack{t^a_{i_k} \in \mathcal{N}(v^a_{i_k}),\\ t^b_{j_k} \in \mathcal{N}(v^b_{j_k})}}{\arg\max} \; |\mathbf{h} \cdot \mathbf{n}(t^a_{i_k})| + |\mathbf{h} \cdot \mathbf{n}(t^b_{j_k})|$$

  where $\mathbf{h}$ is the vector between the barycenters (i.e., the center of mass) of the two triangles, and $\mathbf{n}(t)$ is the normal vector of the triangle t.

- For each of the triangle $t^a_{i_k}$ above, we construct a tetrahedron using one of the three vertices of $t^b_{j_k}$ as the fourth vertex. We also form a tetra hedron for each of the triangles $t^b_{j_k}$.
  - How the fourth vertex is picked is not specified. Perhaps because the authors just want to simplify the implementation.
  - If the triangles $t^a_{i_k}$ and $t^b_{j_k}$ are close to being coplanar, the tetrahedra will be degenerate. In this case, we simply use $t^a_{i_k}$'s and $t^b_{j_k}$'s fourth vertices.
- The energy E_R is defined as

$$E_R = \sum_{a,b} \sum_{k=1}^{|P_{a,b}|} \left(w_S \|\widehat{F}_{t^a_{i_k}} - \widehat{F}_{t^b_{j_k}}\|^2 + w_I \|\widehat{F}_{t^a_{i_k}} - I\|^2 + w_I \|\widehat{F}_{t^b_{j_k}} - I\|^2 \right)$$

  where $\widehat{F}_{t^a_{i_k}}$ is the affine transformation of the newly constructed tetrahedron based on $t^a_{i_k}$.
- The energy is minimized via the same two-phase method described in the Sumner–Popovic paper, which involves a sequence of linear solves.
- In each iteration, after the linear solve, we need to update the set of closest valid vertex pairs.
  - For each vertex of the deformed target mesh, we find its closest point in the source mesh.
  - If the distance between the vertices is less than a user specified threshold and the difference between the vertices' normals is less than 90 degrees, we add the vertex pair to the valid set.
- After the deformation is done, we compute the triangle correspondences.
  - For each deformed target triangle, if one or more of its three vertices are among the user-specified correspondences or in the set of closest valid vertex pairs, it will not be assigned any corresponding source triangles.
    - Why do this?
  - Otherwise, the corresponding source triangle for that target triangle is the source triangle whose barycenter is the closest to the barycenter of the target triangle.

4.3 Deformation Transfer

- After we have the corresponding triangles, we can solve for the deformed vertex positions $\widetilde{\mathbf{v}} = (\widetilde{\mathbf{v}}_1, \widetilde{\mathbf{v}}_2, \ldots, \widetilde{\mathbf{v}}_m)$.
- We minimize a 4-term energy function.

$$E = w_1 E_1 + w_2 E_2 + w_3 E_3 + w_4 E_4$$

  The paper uses $w_1 = w_2 = w_4 = 1$ and $w_3 = 0.1$.

  - Can't they come up with better notations?

- The E_1 term measures the difference between the source and the target transformations.

$$E_1 = \sum_{i=1}^{M} \|F_{s_i} - F_{t_i}\|^2.$$

- The E_2 term forces the transformations of the orphan triangles to be close to the transformation of its adjacent triangles.

$$E_2 = \sum_{t_i \in H} \sum_{t_j \in \mathcal{N}(t_i)} \|F_{t_i} - F_{t_j}\|^2.$$

- E_3 term maintains the spatial relationship between components by preserving the edge lengths of the vertex pairs in each $P_{a,b}$.

$$E_3 = \sum_{T_a, T_b} \sum_{k=1}^{|P_{a,b}|} \left(\|\widetilde{\mathbf{v}}^a_{i_k} - \widetilde{\mathbf{v}}^b_{j_k}\| - \|\mathbf{v}^a_{i_k} - \mathbf{v}^b_{j_k}\| \right)^2$$

  - Note that $\|\mathbf{v}^a_{i_k} - \mathbf{v}^b_{j_k}\|$ is computed on the rest pose of the target object and remains constant.
  - I think this loss function is not quadratic, so we cannot optimize it by a linear solve.

- The E_4 term tries to preserve the surface details of orphan components. It is defined as:

$$E_4 = \sum_{T_a \in H} \left\| L_{T_a} \widetilde{\mathbf{v}}_{T_a} - \widehat{\delta}(\widetilde{\mathbf{v}}_{T_a}) \right\|^2.$$

  where

  - $\widetilde{\mathbf{v}}_{T_a}$ is the vertex positions of the vertices in T_a in the deformed target mesh,
  - L_{T_a} is the Laplacian operation matrix computed on the mesh T_a, and
  - $\widehat{\delta}(\widetilde{\mathbf{v}}_{T_a})$ is the Laplacian coordinates of the vertices in T_a [HSL+06]:

$$\widehat{\delta}(\widetilde{\mathbf{v}}_{T_a}) = \frac{L_{T_a} \widetilde{\mathbf{v}}_{T_a}}{\|L_{T_a} \widetilde{\mathbf{v}}_{T_a}\|} \|\widetilde{\mathbf{v}}_{T_a}\|.$$

- Because E_3 and E_4 are not quadratic functions of the vertex positions, we need to optimize for E with a more general non-linear least squares algorithm.

  - The paper says a Gauss–Newton method should be used, following the work of Shi et al.[SZT+07].
  - However, I think we can use Levenberg–Marquardt [Khu24] too.

5 Discussion

- The algorithm in the paper is much more complicated than the algorithm in the classic paper.
- Most of the paper is dry description of the algorithm, which makes it kind of hard to read, but rather concise. (Maybe too concise.)
- The loss terms E_4 in the last section is not needed if there are no orphan components.
  - Because of this, I'm not reading about the Laplace operator
- E_2 is probably always needed because there will always be orphan triangles (because the paper explicitly omits adding some triangles to the set of correspondences).
- Most of the results given in the paper involve transferring deformation from a single-component source to a multiple-component target. There is only one example where the source is also multi-component.
- The paper mentions that the algorithm works with a multi-component source *as is*, but I don't think this is true.
  - The paper never specifies whether we need to construct the component graph of the source mesh, and how it is mapped to the target mesh.
  - In case parts move relative to each other, the identity of the closest vertex pairs might change. How is one supposed to handle this?
- A later paper points out the algorithm makes an assumption that the special relationships between the parts are the same as the rest pose [HKT10].
  - This is encoded by the E_3 term.
  - This makes it unsuitable for movements where the spatial relationships between the parts would change like in animation whether body parts move relative to one another.

References

[BS08] M. Botsch and O. Sorkine, *On linear variational surface deformation methods*, IEEE Transactions on Visualization and Computer Graphics **14** (2008), no. 1, 213–230.

[BSPG06] M. Botsch, R. W. Sumner, M. Pauly, and M. Gross, *Deformation transfer for detail-preserving surface editing*, Vision, modeling, and visualization 2006 : proceedings, VMV '06, Akademische Verlagsgesellschaft, 2006, pp. 357–364.

[HKT10] E. S. L. Ho, T. Komura, and C.-L. Tai, *Spatial relationship preserving character motion adaptation*, ACM Trans. Graph. **29** (2010), no. 4.

[HSL+06] J. Huang, X. Shi, X. Liu, K. Zhou, L.-Y. Wei, S.-H. Teng, H. Bao, B. Guo, and H.-Y. Shum, *Subspace gradient domain mesh deformation*, ACM Trans. Graph. **25** (2006), no. 3, 1126–1134.

[Khu24] P. Khungurn, *Algorithms for non-linear least squares*, `https://pkhungurn.github.io/notes/notes/scicomp/nonlinear-least-square-algo/nonlinear-least-square-algo.pdf`, 2024.

[SCOL+04] O. Sorkine, D. Cohen-Or, Y. Lipman, M. Alexa, C. Rössl, and H.-P. Seidel, *Laplacian surface editing*, Proceedings of the 2004 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing (New York, NY, USA), SGP '04, Association for Computing Machinery, 2004, pp. 175–184.

[SP04] R. W. Sumner and J. Popović, *Deformation transfer for triangle meshes*, ACM Trans. Graph. **23** (2004), no. 3, 399–405.

[SZT+07] X. Shi, K. Zhou, Y. Tong, M. Desbrun, H. Bao, and B. Guo, *Mesh puppetry: cascading optimization of mesh deformation with inverse kinematics*, ACM SIGGRAPH 2007 Papers (New York, NY, USA), SIGGRAPH '07, Association for Computing Machinery, 2007, p. 81–es.

[YZX+04] Y. Yu, K. Zhou, D. Xu, X. Shi, H. Bao, B. Guo, and H.-Y. Shum, *Mesh editing with poisson-based gradient field manipulation*, ACM Trans. Graph. **23** (2004), no. 3, 644–651.

[ZRKS05] R. Zayer, C. R ossl, Z. Karni, and H.-P. Seidel, *Harmonic guidance for surface deformation*, Computer Graphics Forum **24** (2005), no. 3, 601–609.

[ZXTD10] K. Zhou, W. Xu, Y. Tong, and M. Desbrun, *Deformation transfer to multi-component objects*, Computer Graphics Forum **29** (2010), no. 2, 319–325.